



Администратор

Linux

Vagrant



Проверить, идет ли запись

Меня хорошо видно & слышно?



Ставим "+", если все хорошо
"-", если есть проблемы



Тема вебинара

Vagrant. Настройка рабочего окружения.



Пахомов Григорий

Разработчик в Aquarius

- Реализовал систему автосертификации hw/sw под ОС Astra Linux;
- Автор статей на Хабре;
-  - pcade

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе
#канал группы



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Маршрут вебинара

Знакомство

Теория

- Прикладное применение
- Терминология и определения
vagrant cloud, boxes, провайдеры

Практическая работа

- установка
- управление виртуальной машиной из командной строки `vagrant`
выбор бокса, выбор провайдера

Практическая работа с Vagrantfile

- определение ресурсов
выделение дисков, настройка сети, проброс портов, синхронизация папок
- Провижининг
- single VM и multi VM
- использование конструкций языка Ruby
переменные, циклы, условия

Рефлексия

Цели вебинара

К концу занятия вы сможете

1.	Настроить окружение на своем хосте
2.	Повторить приемы, которые использовали на занятиях
3.	Прочитать, проанализировать Vagrantfile
4.	Описать конфигурацию виртуальных машин с помощью Vagrantfile
5.	Запустить виртуальную машину



Смысл

Зачем вам это уметь

ВАШ ВАРИАНТ?

Смысл

Зачем вам это уметь

- | | |
|----|--|
| 1. | Легко и просто выполнить домашнее задание. А также проверять. |
| 2. | Прикладной пример практики Infrastructure as a Code (IaC) для хранения описания конфигурации виртуальных машин в репозитории git |
| 3. | Экономия времени, в сравнении с ручным созданием виртуальных машин. Повторяемость (но возможны отклонения от задуманного) |
| 4. | Vagrant часто применяется в разработке и тестировании |



**Что вы знаете о Vagrant?
Кто уже настроил рабочее
окружение?**

Настраиваем окружение для работы

Описание виртуальной машины в Vagrantfile

The image displays a virtual machine configuration window on the left and a Vagrantfile code editor on the right. Red arrows indicate the mapping between the GUI settings and the code.

Virtual Machine Configuration (Left Panel):

- General:** Name: task1_vm-1_1714997691011_21573, Operating System: Red Hat (64-bit)
- System:** Base Memory: 2048 MB, Boot Order: Hard Disk, Optical, EFI: Enabled, Acceleration: Nested Paging, PAE/NX, KVM Paravirtualization
- Display:** Video Memory: 4 MB, Graphics Controller: VBoxVGA, Remote Desktop Server Port: 5955, Recording: Disabled
- Storage:** Controller: IDE Controller, Controller: SATA Controller, SATA Port 0: packer-almalinux-9-1700337286-disk01.vmdk (Normal, 19.53 GB)
- Audio:** Disabled
- Network:** Adapter 1: Intel PRO/1000 MT Desktop (NAT), Adapter 2: Intel PRO/1000 MT Desktop (Host-only Adapter, 'VirtualBox Host-Only Ethernet Adapter #8')

Vagrantfile Code (Right Panel):

```
Vagrant.configure(2) do |config|
  config.vm.define "vm-1" do |srv|
    srv.vm.box = "alma/9.3"
    srv.vm.synced_folder ".", "/vagrant"
    srv.vm.hostname = "srv-1"
    srv.vm.network(:private_network, ip: "192.168.50.11")
    srv.vm.provider "virtualbox" do |vb|
      vb.memory = "2048"
      vb.cpus = "1"
    end
  end
end
```

Arrows indicating mappings:

- From **Base Memory: 2048 MB** to `vb.memory = "2048"`
- From **Adapter 1: Intel PRO/1000 MT Desktop (NAT)** to `srv.vm.network(:private_network, ip: "192.168.50.11")`



Почему выбрали

IaC Infrastructure as a Code -

- храним описание инфраструктуры в git репозитории
- повторяемость
 - используем для демо стендов
 - используем для проверки домашних
- достаточно локального компьютера (без доступа в cloud)
 - для production - используйте Terraform, k8s или другое
- переносимость Win, Linux, MacOS (ARM based M[123] ?)



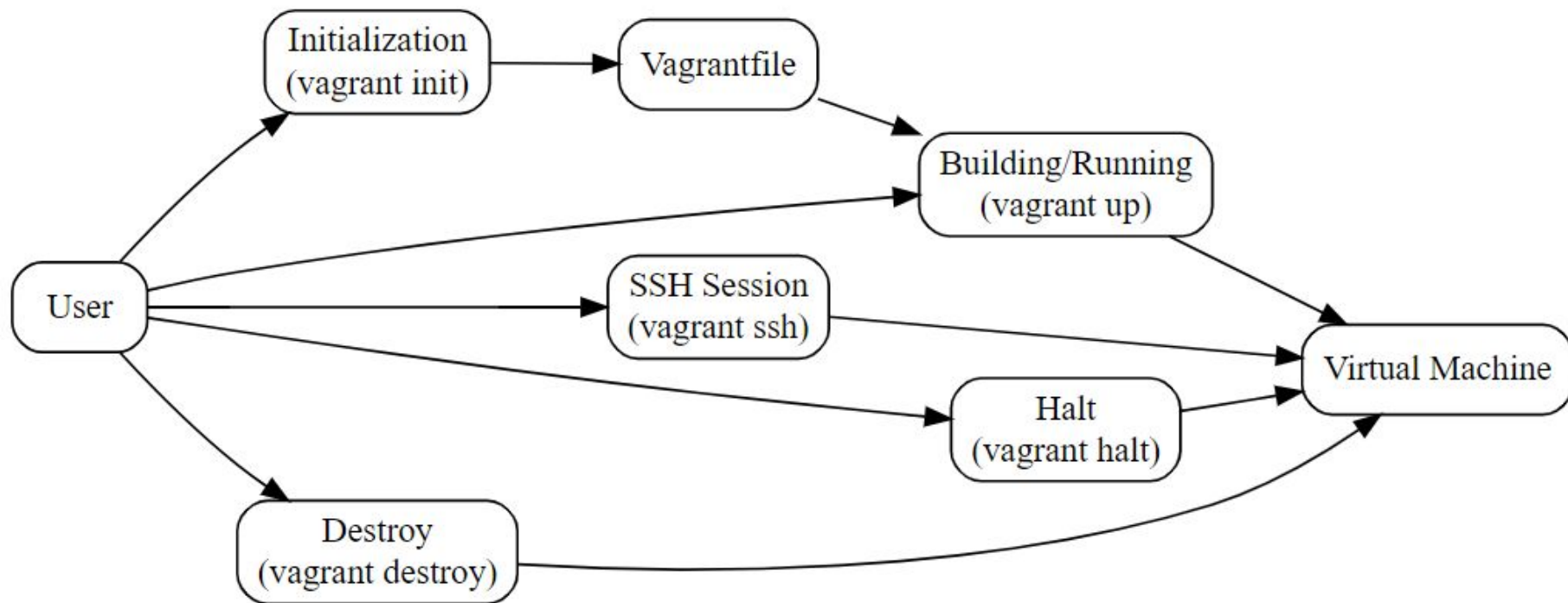
Терминология для работы с Vagrant

1. Среда виртуализации - **provider** (Virtualbox, Vmware, Hyper-V)
2. Образ виртуальной машины- (**box**) файл с установленным дистрибутивом. Диск виртуальной машины (VM image).
3. **Vagrantfile** файл с описанием конфигурации виртуальной машины.
4. **Виртуальная машина** - копия образа, созданная после старта виртуальной машины.
5. **vagrant** - утилита командной строки
6. **Provisioning** - процесс настройки виртуальной машины после старта

Процесс настройки для работы в окружении Vagrant

1. git - создайте себе аккаунт на github
2. установите среду виртуализации provider Virtualbox,
3. установите vagrant, packer - инструмент создания образов (опционально),
4. написать Vagrantfile или загрузить проект с описанием конфигурации виртуальных машин
5. создаем образ (image) box
6. стартуем виртуальные машины
7. добавляем код для установки, настройки проекта (bash, ansible или что-то другое) provisioning (опционально)

Шаги в сессии работы vagrant и команды



Примеры файлов для занятия

<https://github.com/pcade/vagrant-examples>

Базовые шаги

Настраиваем минимальное рабочее окружение

Загружаем образ

Создаем Vagrantfile

Добавляем памяти

Старт остановка виртуальной машины

Выбираем тип виртуализации

Создаем файл Vagrantfile

Файл можно создать

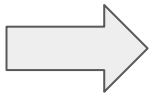
- вручную (копия существующего) либо
- сгенерировать командой **vagrant init**

по умолчанию в файле есть описание основных настроек

vagrant init -minimal - файл без описания

vagrant init -force - перепишет файл

```
vagrant init -f  
vagrant init -m -f
```



```
Vagrant.configure("2") do |config|  
  config.vm.box = "base"  end
```

Находим и загружаем готовый образ виртуальной машины

Сервис от vagrant <https://app.vagrantup.com/boxes/search>

Загружаем box командой `vagrant box` (может потребоваться VPN)

```
vagrant box add ubuntu/bionic64
==> box: Loading metadata for box 'ubuntu/bionic64'
      box: URL: https://vagrantcloud.com/api/v2/vagrant/ubuntu/bionic64
==> box: Adding box 'ubuntu/bionic64' (v20230607.0.5) for provider: virtualbox
      box: Downloading:
https://vagrantcloud.com/ubuntu/boxes/bionic64/versions/20230607.0.5/providers/virtualbox/unknown/vagrant.box
      box:
==> box: Successfully added box 'ubuntu/bionic64' (v20230607.0.5) for 'virtualbox'!
```

Варианты получения образа box виртуальной машины

1. Импортировать из других форматов (VDI, VMDK, QCOW);
2. Создаем образ инструментом packer:
 - Используем инструмент packer;
 - Готовые шаблоны сборки;
 - Создаём собственный шаблон;
3. Используем готовый образ:
 - Находим нужный образ в интернете;
 - Загружаем box командой `vagrant box` (может потребоваться VPN);
 - Загружаем box вручную (`curl`, `wget`, браузер) и импортируем образ командой `vagrant box` из файл;

Находим и загружаем готовый образ виртуальной машины

Загружаем box вручную (curl, wget, браузер) и импортируем образ командой `vagrant box` из файла (без VPN)

Находим URL образа

- https://cloud.centos.org/centos/9-stream/x86_64/images/
- https://dl.rockylinux.org/pub/rocky/9/images/x86_64/
- <http://cloud-images.ubuntu.com/releases/23.04/release/>

Примеры названий образа

`ubuntu-23.04-server-cloudimg-amd64-vagrant.box`

`Rocky-9-Vagrant-Vbox-9.6-20250531.0.x86_64.box`

`Tumbleweed.x86_64-1.0-virtualbox-Snapshot20250410.vagrant.virtualbox.box`

[Инструкция по импорту образа без VPN](#)

Пример загрузки образа

```
Usage: vagrant box add [options] <name, url, or path>
```

Расположение образа можно задать

- name имя box в HashiCorp's Vagrant Cloud (нужен VPN),
- URL прямая ссылка на файл .box,
- путь к локальному файлу .box, локальный .json файл с метаданными.

```
curl -L -O 'https://app.vagrantup.com/bento/boxes/debian-12.1/versions/202309.08.0/providers/virtualbox.box'  
vagrant box add virtualbox.box --name debian/12.1  
vagrant box list
```

Vagrantfile: single vm

```
# пример минимального файла
Vagrant.configure(2) do |config|    # объект конфигурации
  config.vm.define "vm-1" do |srv|  # объект виртуальной машины
    srv.vm.box = "debian/12.1"    # задали box (VM image)
  end
end
```

Размера памяти по умолчанию может оказаться недостаточно

```
Vagrant.configure(2) do |config|
  config.vm.define "vm-1" do |srv|
    srv.vm.box = "debian/12.1"
    srv.vm.provider "virtualbox" do |vb|
      vb.memory = "2048"          # увеличили размер памяти в VM
    end
  end
end
```

После того как создан файл с описанием конфигурации можно запускать виртуальную машину

Команды управления виртуальной машиной

```
vagrant up # старт и создание VM из файла описания
vagrant status # проверим состояние
vagrant global-status # проверим состояние всех VM
```

```
vagrant ssh # подключение к VM
vagrant ssh name # подключение к VM по имени
vagrant ssh-config # приватный ключ, настройки подключения
```

```
vagrant reload # перезагрузка
vagrant halt # остановка
vagrant destroy # удаление VM вместе с данными приложений
```

Выбор системы виртуализации – провайдера

выбрать провайдера можно:

1. в файле Vagrantfile используя *config.vm.provider*
2. в командной строке *vagrant up --provider*
3. *vmware_fusion*
в переменной окружения *VAGRANT_DEFAULT_PROVIDER*

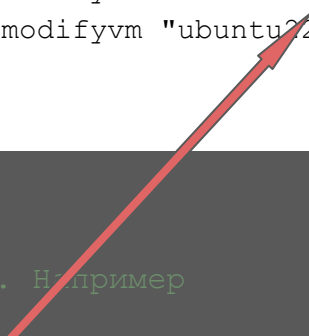
```
config.vm.provider "virtualbox"      # выбор provider для объекта conf
config.vm.provider "vmware_fusion"
config.vm.define "vm-1" do |srv|
  srv.vm.provider "virtualbox" do |vb| # настройки для объекта vm
    vb.memory = "2048"
  end
end
```

Поддерживаемая конфигурация и доступные настройки зависят от типа провайдера

Настройки под определенного провайдера

- [Настройки под Vmware](#)
- [Настройки под Hyper-V](#)
- [Настройки для VirtualBox](#)

```
VBoxManage modifyvm "ubuntu2204" -audio none
VBoxManage modifyvm "ubuntu2204" -nested-hw-virt on
VBoxManage modifyvm "ubuntu2204" --natdnshostresolver1
on
```



```
config.vm.define "vm-1" do |srv|
  srv.vm.provider "virtualbox" do |vb|
    # для некоторых настроек есть сокращенный вариант. Например
    vb.check_guest_additions = false
    vb.memory = "512"
    vb.name = "my_vm" # имя виртуальной машины
    # "под капотом" customize выполняет команду vboxmanage
    vb.customize ['modifyvm', :id, '--nested-hw-virt', 'on']
    vb.customize ['modifyvm', :id, '--audio', 'none']
  end
end
```

Вопросы?



Ставим "+",
если вопросы есть



Ставим "-",
если вопросов нет

Настраиваем Vagrant окружение

Взаимодействие виртуальных машин
со внешним миром

Внешние директории

Типы сетевых подключений

Добавляем диски

Создаем несколько виртуальных
машин

Провижининг

Синхронизация директорий

Документация

- vboxfs - virtualbox (default) нужны драйвера в гостевой системе
- nfs
- rsync
- smb

```
config.vm.synced_folder ".", "/vagrant", owner: "vagrant",  
group: "vagrant", mount_options: ["uid=1234", "gid=1234"]
```

Диски

```
export VAGRANT_EXPERIMENTAL = "disks" # выполнить перед vagrant up
```

```
config.vm.define "server" do |server|  
  server.vm.provider "virtualbox" do |vb|  
    vb.memory = "1024"  
  end  
  server.vm.disk :disk, size: "1GB", name: "disk1"  
  server.vm.disk :disk, size: "1GB", name: "disk2"  
  server.vm.disk :disk, size: "1GB", name: "disk3"
```

Настройка сети

[vagrant-examples/network/Vagrantfile at master · nixuser/vagrant-examples](#)

```
box.vm.hostname = "myhost"
# Проброс одного порта. Порт доступен снаружи.
box.vm.network "forwarded_port", guest: 22, host: 3022

# Сеть типа Host-only. Доступна только из VM либо соседней VM в той же сети
box.vm.network( "private_network", ip: '192.168.50.11',
intnet: "otus-net") # имя сети

# Сеть типа bridge. Доступна снаружи.
box.vm.network("public_network", ip: "192.168.0.17")
```

⇒ vm-1: Preparing network interfaces based on configuration...

vm-1: Adapter 1: nat

vm-1: Adapter 2: intnet

vm-1: Adapter 3: bridged

⇒ vm-1: Forwarding ports...

vm-1: 80 (guest) ⇒ 8888 (host) (adapter 1)

vm-1: 22 (guest) ⇒ 2200 (host) (adapter 1)

Network

Adapter 1: Intel PRO/1000 MT Desktop (NAT)

Adapter 2: Intel PRO/1000 MT Desktop (Internal Network, 'otus-net')

Adapter 3: Intel PRO/1000 MT Desktop (Bridged Adapter, Intel(R) Wi-Fi 6 AX201 160MHz)

Multi VM

- декларативное описание каждой VM может быть трудоемко

```
Vagrant.configure("2") do |config|
  config.vm.box = "almalinux/9.2" # общие настройки для VM

  config.vm.define "web" do |web|
    web.vm.hostname = "apache" # настройка для VM
  end

  config.vm.define "db" do |db|
    db.vm.hostname = "mysql" # настройка для VM
  end
end
```

- использование программных конструкций для перебора в цикле

Provisioning

```
# запуск команд из встроенных скриптов
srv.vm.provision type: "shell", name: "mem", inline:
"free -m"
srv.vm.provision "shell", inline: <<-SHELL
    uname -a
    free -m
SHELL
# запуск команд из файла
srv.vm.provision "shell", path: "install_nginx.sh"
```

```
# Скрипты выполняются в случае запуска команд
vagrant up
vagrant provision
vagrant provision --provision-with x,y,z
vagrant reload --provision # стоп, старт VM
```

Провижининг – это запуск команд после создания и старта VM

- Установка пакетов

- Создание пользователей

- Добавление файлов

Вопросы?



Ставим "+",
если вопросы есть



Ставим "-",
если вопросов нет



Практика

Вспомнить пропущенные методы, атрибуты

В чат пишем номер строки и пропущенное слово, либо идею о том, что это.

```
1. Vagrant.__(2) do |config|
2.   config.vm.__ "vm-1" do |srv|
3.     srv.vm.__ = "almalinux/9.2"
4.     srv.vm.__ "./", "/vagrant"
5.     srv.vm.__ = "srv-1"
6.     srv.vm.__ (:private_network, ip: "192.168.50.11")
7.     srv.vm.__ "virtualbox" do |vb|
8.       vb.__ = "2048"
9.       vb.__ = "1"
10.    end
11.  end
12.end
```

Заполненный Vagrantfile

```
1. Vagrant.configure(2) do |config|
2.   config.vm.define "vm-1" do |srv|
3.     srv.vm.box = "almalinux/9.2"
4.     srv.vm.synced_folder "./", "/vagrant"
5.     srv.vm.hostname = "srv-1"
6.     srv.vm.network(:private_network, ip: "192.168.50.11")
7.   srv.vm.provider "virtualbox" do |vb| 8
vb.memory = "2048"
9     vb.cpus = "1"
10.   end
11. end
12. end
```

Advanced topics

Оптимизация работы

Уменьшаем количество строк в
Vagrantfile

Используем условные операторы
Разбираем пример добавления
контроллеров и дисков

Словари и цикл

- multi VM - больше двух
- отделяем описание VM от кода.
- для добавления VM - добавить одну строку.

```
# vagrant_examples/program_constructions1/
nodes = {
  :pcm1 => { :ip_addr => '192.168.7.150' },
  :pcm2 => { :ip_addr => '192.168.7.151' },
}

Vagrant.configure(2) do |config| nodes.each do |boxname, boxconfig|
  # настройки виртуальной машины
  config.vm.define boxname do |box| box.vm.hostname = boxname.to_s # pcm1, pcm2
  box.vm.network("private_network", ip: boxconfig[:ip_add
end
end
```

Условия

```
# выполняет код если условие false
unless File.exist?(dconf[:dfile])
  vb.customize ['createhd', '--filename',
    dconf[:dfile], '--variant', 'Fixed', '--size', dconf[:size]]
end
```

```
if boxconfig.key?(:public) # выполняет код если ключ определен
box.vm.network "public_network", boxconfig[:public] end
```

```
case boxname.to_s # замена множества if
when "inetRouter"
  ...
when "centralRouter"
  ...
end
```

Добавление контроллера и дисков.Простая v..

```
vb.customize ['createhd',  
  '--filename', '../sata1.vdi',  
  '--variant', 'Fixed',  
  '--size', '250']  
vb.customize ['storageattach', :id,  
  '--storagectl', 'SATA Controller',  
  '--port', 1,  
  '--device', 0,  
  '--type', 'hdd', '--medium', '../sata1.vdi']
```

Недостатки: выполнить `vagrant halt`; `vagrant up`

```
VBoxManage.exe list hdds  
VBoxManage.exe createhd --filename ../sata1.vdi  
--variant Fixed --size=100  
VBoxManage.exe closemedium disk 4a746d27-ac29-49ae-9371-ab45facc6  
--delete  
VBoxManage.exe showvminfo 0e978413-c42a-4e55-a891-a41a76298e9e
```

Добавление контроллера и дисков

```
box.vm.provider :virtualbox do |vb|  
  # проверка на существование файла диска и # необходимость создания контроллера  
  needsController = false  
  unless File.exist?(' ../sata1.vdi')  
    vb.customize ['createhd', '--filename', ' ../sata1.vdi', '--variant', 'Fixed', '--size',  
      '250']  
    needsController = true  
  end
```

```
  if needsController == true  
    vb.customize ["storagectl", :id,  
      "--name", "SATA", "--add", "sata" ]  
    vb.customize ['storageattach', :id,  
      '--storagectl', 'SATA',  
      '--port', 1,  
      '--device', 0,  
      '--type', 'hdd', '--medium', ' ../sata1.vdi']
```

Список материалов для изучения

Отладка

[Интерактивный ruby для отладки](#)

[Отладка vagrant](#)

Примеры файлов

[Файлы для занятия](#)

[Одна VM](#)

[Несколько VM](#)

[Проект из занятия](#)

Вопросы?



Ставим "+",
если вопросы есть



Ставим "-",
если вопросов нет

Слайд с домашним заданием

1. Получить навыки работы с Git, Vagrant;

2. Обновлять ядро в ОС Linux.

Программа минимум: установить свежее ядро.

Уже настроили рабочее место? Если еще нет - используйте [инструкцию](#)

Для выполнения домашнего задания используйте [методичку](#)

1. Выполните действия, описанные в методичке.
2. Полученный в ходе выполнения ДЗ Vagrantfile и отчет залейте в ваш git-репозиторий.
3. Пришлите ссылку на него в чате для проверки ДЗ. Обычно мы проверяем ДЗ в течение 48 часов.

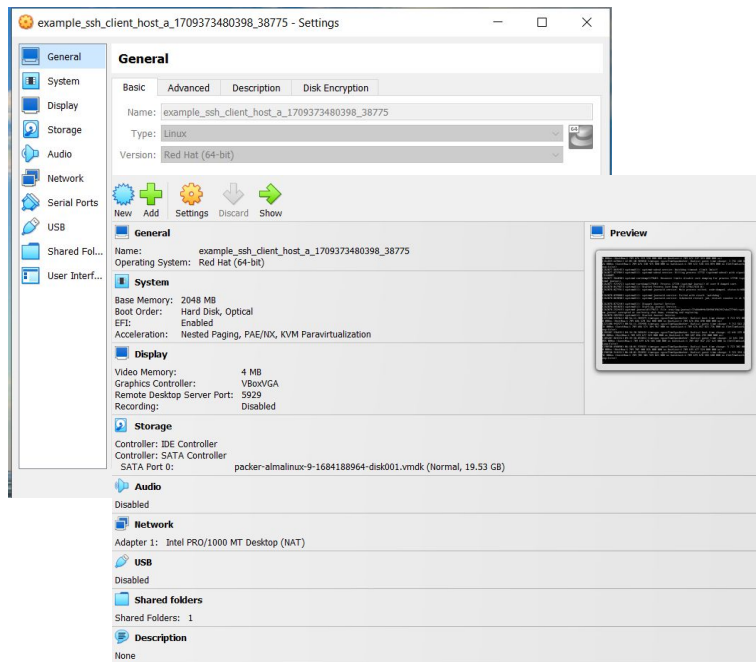
Для выполнения ДЗ со * и ** вам потребуется сборка ядра и модулей из исходников.

Если возникнут вопросы, обращайтесь к студентам, преподавателям и наставникам в канал группы в Telegram.

Удачи при выполнении!

Рефлексия

GUI или командная строка?



```
# -*- mode: ruby -*-
# vi: set ft=ruby :

Vagrant.configure("2") do |config|

  config.vm.define "demo" do |server|
    # config.vm.box = "debian/11"
    config.vm.box = "almalinux/9"

    config.vm.provider "virtualbox" do |vb|
      # Customize the amount of memory on the VM:
      vb.memory = "4096"
      vb.customize ['modifyvm', :id, '--nested-hw-virt', 'on']
    end

    #server.vm.provision "shell",
    #  name: "config",
    #  ~\workdir\vm\lessons\python> vagrant up
    end
    ⇒ demo: Machine already provisioned. Run 'vagrant provision' or use the '--provision'
    end
    ⇒ demo: flag to force provisioning. Provisioners marked to run always will still run.
    ~\workdir\vm\lessons\python>
    ~\workdir\vm\lessons\python> vagrant status
    Current machine states:

    demo                          running (virtualbox)

    The VM is running. To stop this VM, you can run `vagrant halt` to
    shut it down forcefully, or you can run `vagrant suspend` to simply
    suspend the virtual machine. In either case, to restart it again,
    simply run `vagrant up`.
    ~\workdir\vm\lessons\python> |
```

Тест

Вопросы для проверки

Какой файл определяет конфигурацию виртуальной машины в Vagrant?

- A. Dockerfile
- B. Vagrantsetup
- C. Vagrantfile
- D. VMconfig



Отправьте в чат верную букву

Вопросы для проверки

Какой файл определяет конфигурацию виртуальной машины в Vagrant?

- A. Dockerfile
- B. Vagrantsetup
- C. Vagrantfile**
- D. VMconfig

Вопросы для проверки

Что такое Box в Vagrant?

- A. Скрипт для провижининга
- B. Образ виртуальной машины
- C. Файл логов
- D. Конфигурация сети



Отправьте в чат верную букву

Вопросы для проверки

Что такое Box в Vagrant?

- A. Скрипт для провижининга
- B. Образ виртуальной машины**
- C. Файл логов
- D. Конфигурация сети

Вопросы для проверки

Какую среду виртуализации (Provider) необходимо установить на хост-машине до установки самой утилиты Vagrant?

- A. Packer
- B. Ansible
- C. Virtualbox (или Vmware, Hyper-V)
- D. Git



Отправьте в чат верную букву

Вопросы для проверки

Какую среду виртуализации (Provider) необходимо установить на хост-машине до установки самой утилиты Vagrant?

A. Packer

B. Ansible

C. Virtualbox (или Vmware, Hyper-V)

D. Git

Вопросы для проверки

Какой механизм в Vagrant отвечает за автоматическую настройку виртуальной машины после запуска?

- A. Provisioning
- B. Forwarding
- C. Syncing
- D. Deploying



Отправьте в чат верную букву

Вопросы для проверки

Какой механизм в Vagrant отвечает за автоматическую настройку виртуальной машины после запуска?

A. Provisioning

B. Forwarding

C. Syncing

D. Deploying

Вопросы для проверки

Что позволяет сделать Multi-VM конфигурация?

- A. Запустить Vagrant без провайдера
- B. Настраивать только сети и порты
- C. Определять и управлять несколькими VM в одном Vagrantfile
- D. Автоматически обновлять версии боксов



Отправьте в чат верную букву

Вопросы для проверки

Что позволяет сделать Multi-VM конфигурация?

- A. Запустить Vagrant без провайдера
- B. Настраивать только сети и порты
- C. Определять и управлять несколькими VM в одном Vagrantfile**
- D. Автоматически обновлять версии боксов

Цели вебинара

Проверка достижения целей

1.	Настроить окружение на своем хосте
2.	Повторить приемы, которые использовали на занятиях
3.	Прочитать, проанализировать Vagrantfile
4.	Описать конфигурацию виртуальных машин с помощью Vagrantfile
5.	Запустить виртуальную машину

Рефлексия



С какими впечатлениями уходите с вебинара?



Как будете применять на практике то, что узнали на вебинаре?

**Заполните, пожалуйста,
опрос о занятии
по ссылке в чате**